SQLST ÜRÜN AİLESİ · İKİNCİ ÜRÜN

KDS — Analiz Dokümanı (00 · çalışma adı)

Web tabanlı, SaaS, self-servis raporlama ve karar destek sistemi — sürükle-bırak sorgu tasarımı + ham SQL + grafik/dashboard

v1 analizi · 4 Ağustos 2026 · İş sırası: docs/roadmap.md (TEK takip yeri)

1. Vizyon

> Müşterinin verisi üzerinde, **SQL bilmeyen kullanıcının sürükle-bırakla sorgu tasarlayıp** sonucu tablo/grafik/dashboard olarak görebildiği; SQL bilen kullanıcının da aynı yerde ham SQL çalıştırabildiği, çok kiracılı bir raporlama ürünü.

Turnusol testi: *"Bu rapor işi için IT'ye/geliştiriciye gitmem gerekir mi?"* — v1'e girecek her özellik bu sorunun cevabını "hayır"a yaklaştırmalı.

SQLST ile ilişki (kullanıcı kararı, 2026-08-04): tamamen **ayrı ürün** — kod paylaşımı yok. Taşınan şey desen bilgisidir: lehçe soyutlaması (SQLST v3 deneyimi), lisans denetimi geleneği, roadmap/dilim disiplini, "yarım özellik yok → gizlenir" ilkesi.

2. Kesinleşen kararlar (2026-08-04, kullanıcı)

#	Karar	Not
K1	Yalnız **SaaS**; on-prem kurulum yok	Tenant mimarisi ve barındırmayı bu belirler (§8, §10)
K2	v1 hedef motor yalnız **MSSQL**	SQL üretimi lehçe arkasında — çok-motor kapısı açık kalır (§5)
K3	v1 kapsamı tam: tablo sonuç + export + grafik + **dashboard**	§11
K4	Backend **.NET 10 LTS**, CQRS + **MediatR 12.x**	v13+ ticari lisansa geçti; **12.5.0'a sabitle, [12.*,13) üst sınır** (Apache 2.0)
K5	**EF Core + ince UnitOfWork yalnız metadata DB'de**	DbContext zaten UoW; generic repository töreni yok (§6)
K6	Klasik katman: **Api / Business / Dal (+ Contracts)**	Ekip alışkanlığıyla uyumlu; Business = CQRS handler'ları
K7	Frontend **React + TypeScript**	§9
K8	Kimlik: **Keycloak** (OIDC)	Ayrı ürün olsa da ailede SSO potansiyeli açık kalır
K9	Sorgu = **JSON modeli**; SQL üretimi backend'de	§5
K10	**Veri haritası** (eski adı: semantik katman / veri modeli) + ham SQL editörü birlikte	§4
K11	**İki çalıştırma modu**: Canlı + İçe Alım, config'den seçilir	§3

3. Çalıştırma modları — "hedef DB'de mi, kendi DB'me çekip mi?"

Kullanıcı sorusu (2026-08-04): *"İki özelliği de koysak, config'den seçsek: hedef DB'de çalıştır ya da bir job'la kendi DB'me çekip orada çalıştır — olabilir mi?"*

> **Cevap: evet, olur ve sektörde birebir karşılığı var — Power BI'daki DirectQuery vs Import ayrımı.** İki mod da v1'e girer; **bağlantı düzeyinde varsayılan + veri haritası düzeyinde geçersiz kılma** ile seçilir.

	Mod A · (DirectQuery)	Mod B · (Import)
Sorgu nerede koşar	Müşterinin hedef DB'sinde	KDS'nin kendi deposunda (tenant DB'si)
Veri tazeliği	Anlık	Son senkron anı (zamanlanmış job)
Kaynak DB'ye yük	Her rapor açılışında	Yalnız senkron anında
Depolama maliyeti	Yok	Var (tenant deposu büyür)
Rapor hızı	Kaynağa ve ağa bağlı	Öngörülebilir, genelde hızlı
İyi uyduğu yer	Operasyonel/anlık raporlar	Yoğun dashboard'lar, geçmiş analizi, yavaş/uzak kaynaklar

SaaS'ın asıl zorluğu — müşteri DB'sine erişim

Müşteri MSSQL'i genelde şirket ağındadır; bizim bulutumuz ona kendiliğinden ulaşamaz.

- **v1 varsayımı:** hedef DB SaaS'tan erişilebilirdir (Azure SQL / internete açık MSSQL / IP whitelist / müşteri VPN'i). Ürün belgeleri bunu kurulum önkoşulu yazar.
- **KDS Ajanı** (müşteri ağına kurulan, dışarı doğru bağlantı açan hafif gateway — Power BI On-premises Data Gateway modeli) **v1 DIŞI**, araştırma kuyruğunda.
- Mod B bu problemi hafifletir: erişim yalnız senkron job anında gerekir; ileride ajan gelirse aynı job müşteri tarafında koşup veriyi **itme (push)** modeliyle gönderir — mimari bugünden buna göre çizilir (senkron işi tek modülde toplanır).

İçe alım hattı (v1): Hangfire job → veri haritasında seçilen tablolar → tenant'ın kendi deposuna **tam yükleme** (truncate+load) → sorgular orada koşar. Zamanlama (cron), son senkron durumu/süresi/satır sayısı UI'da. Artımlı yükleme (rowversion/CDC) v1 DIŞI — araştırma kuyruğu.

4. Veri haritası (eski adı: semantik katman)

Varlık zinciri: **Bağlantı → Veri Haritası → Sorgu/Rapor → Dashboard**

- **Bağlantı:** sunucu + DB + salt-okunur kimlik; şifre şifreli saklanır (§7).
- **Veri Haritası:** tasarımcının seçtiği tablolar; dostça adlar (`FKTHR_MSTR` → "Fatura Ana"); önceden tanımlı ilişkiler (join'ler); kolonlarda **boyut/ölçü** işareti; çalıştırma modu seçimi (canlı / içe alım).
- **Sorgu/Rapor:** JSON sorgu modeli + görselleştirme tercihi; klasörlenir, yetkilenir.
- **Dashboard:** raporların grid yerleşimi + dashboard-düzeyi filtreler.

Son kullanıcı ham tabloya değil veri haritasına bakar; ham SQL ayrı ekranda, ileri kullanıcı rolüne açıktır. KDS'lerde başarıyı bu katman belirler — son kullanıcı join kuramaz, kurmamalı; ilişkiyi harita tanımlar, tasarımcı sadece sürükler.

5. Sorgu temsili ve SQL üretimi

Tasarlanan sorgu **SQL olarak değil, JSON modeli (AST) olarak** saklanır; SQL'e çeviri backend'de, motor lehçesine göre yapılır. Kazanımlar: sorgu versiyonlanabilir/denetlenebilir, UI'dan bağımsız doğrulanabilir, çok-motor kapısı açık kalır.

v1 JSON modeli kapsamı: kaynak varlıklar (modelden) · kolon seçimi · join (modeldeki ilişkilerden) · filtre ağacı (AND/OR, parametreli) · GROUP BY + aggregate (SUM/COUNT/AVG/MIN/MAX) · sıralama · limit/sayfalama.
v1 DIŞI: alt sorgu, UNION, window fonksiyonları → ihtiyaç ham SQL ekranına yönlendirilir.

Hat: React tasarımcı → JSON → doğrulama (şema + model/yetki süzgeci) → `ISorguUretici` (v1: `MssqlSorguUretici`) → **parametreli** SQL → çalıştırıcı.

Kurallar (MersisMSSQL derslerinin mirası): nesne adları daima QUOTENAME mantığıyla parantezlenir; değerler asla birleştirilmez, daima parametre; üretilen SQL kullanıcıya **"SQL'i Göster"** ile sunulur (şeffaflık + öğrenme + hata ayıklama).

6. İki veri dünyası (mimarinin bel kemiği)

	Metadata dünyası	Çalıştırma dünyası
Ne tutar / yapar	Tenant, kullanıcı-yetki, bağlantı, veri haritası, rapor, dashboard, job tanımı, audit	Tasarlanan/yazılan sorguların icrası + şema keşfi
Nerede	KDS'nin kendi DB'si	Müşteri hedef DB'si **veya** tenant içe-alım deposu
Erişim tekniği	EF Core + ince `IUnitOfWork`	Saf ADO.NET (`Microsoft.Data.SqlClient`); EF **asla** girmez
Koruma	Normal uygulama kuralları	Read-only bağlantı, `CommandTimeout`, reader'da satır sınırı, `CancellationToken` ile iptal, audit

Bu ayrım birinci günden proje yapısına yansır: çalıştırma dünyası ayrı modüldür ve metadata dünyasının DbContext'ine referans veremez.

7. Güvenlik

- **SELECT-only bekçisi:** `Microsoft.SqlServer.TransactSql.ScriptDom` (MIT, resmî T-SQL parser) ile **AST doğrulaması** — string kontrolü değil. Yalnız SELECT statement'ları geçer; çoklu statement, DDL/DML/EXEC, `INTO`, hint istismarları red. Hem ham SQL ekranı hem üretilen SQL aynı bekçiden geçer (tek kapı).
- **İkinci katman:** hedef bağlantılar salt-okunur DB kullanıcısıyla kurulur; ürün belgeleri müşteriden read-only login ister (bekçi delinse bile yazamaz).
- **Bağlantı şifreleri:** ASP.NET Data Protection ile şifreli, key ring kalıcı depoda; düz metin hiçbir yerde durmaz (vault entegrasyonu araştırma kuyruğu).
- **Limitler:** satır sınırı + zaman aşımı (tenant/sorgu bazlı ayarlanabilir) + tenant başına eşzamanlı sorgu sınırı.
- **Keycloak:** tek realm; tenant ayrımı organizasyon/grup + token claim'i; kaba roller token'da (**Admin / Geliştirici / Müşteri**); rapor/klasör-bazlı ince yetki metadata DB'de.
- **Audit:** kim, hangi tenant'ta, hangi sorguyu (JSON/SQL), hangi bağlantıda, ne zaman, kaç satır/ms — her icra kaydedilir.

8. Multi-tenancy

- **Metadata DB:** tek DB + her tabloda `TenantId` + EF Core global query filter.
- **İçe alım deposu: tenant başına ayrı DB** — veri izolasyonu, müşteri ayrılınca temiz silme, yedek/geri yükleme kolaylığı, gürültülü komşu sınırı.
- **Keycloak:** tek realm (realm-per-tenant bilinçli DIŞI — yönetim yükü).

9. Teknoloji seçimi (lisans denetimli — SQLST geleneği)

Bileşen	Seçim	Lisans	Not
Runtime	.NET 10 LTS	MIT	
CQRS	**MediatR 12.5.0**	Apache 2.0	⚠️ **v13+ ticari** — pakette `[12.*,13)` üst sınırı zorunlu
Metadata ORM	EF Core	MIT	Yalnız metadata dünyası
SQL istemci	Microsoft.Data.SqlClient	MIT	Çalıştırma dünyası saf ADO.NET
T-SQL parser	Microsoft.SqlServer.TransactSql.ScriptDom	MIT	SELECT-only bekçisi
Doğrulama	FluentValidation	Apache 2.0	
Job/zamanlayıcı	**Hangfire**	LGPL-3.0	SaaS'ta dağıtım yok → uyumlu; hazır dashboard artı. Alternatif: Quartz.NET (Apache 2.0)
Kimlik	Keycloak (konteyner)	Apache 2.0	
Metadata + depo DB	**MSSQL**	—	v1 motoru zaten MSSQL, ekip uzmanlığı; Express 10 GB/DB sınırıyla başlanabilir. Barındırma maliyeti araştırma kuyruğunda
Frontend	React 19 + TypeScript + Vite	MIT	
UI bileşen kiti	**MUI (Material UI)**	MIT	Kullanıcı kararı 2026-08-04
OIDC istemci	react-oidc-context	MIT	`keycloak-js` yerine standart OIDC
Kanvas	React Flow (xyflow)	MIT	QMS org şemasından tanıdık
Filtre kurucu	react-querybuilder	MIT	
Sonuç grid	**TanStack Table + Virtual**	MIT	⚠️ **AG Grid tuzak:** Community MIT ama pivot/server-side model **Enterprise-ücretli**
Grafik	Apache ECharts	Apache 2.0	Tablo/çubuk/çizgi/pasta/sayı kartı v1
Dashboard yerleşimi	react-grid-layout	MIT	
SQL editörü	Monaco Editor	MIT	SQLST/MiniSSMS deneyimi
Excel export	ClosedXML	MIT	Backend'de üretilir
PDF export	**karar bekliyor**	—	⚠️ **QuestPDF Community gelir eşiğine bağlı ($1M)** — v1'de Excel/CSV yeter, PDF araştırma kuyruğu

10. Barındırma ve Kubernetes analizi

> **Karar: v1'de Kubernetes YOK.** Tek VM + Docker Compose: API + Keycloak + MSSQL + React statik dosyaları (+ Hangfire API içinde). SaaS olduğumuz için her şey zaten konteyner imajı — K8s'e geçiş kapısı açık kalır, hiçbir şey yeniden yazılmaz.

K8s'i yeniden değerlendirme sinyalleri (hangisi gelirse araştırma açılır): tek VM'in dikey sınırına dayanmak · SLA gereği çoklu-düğüm/sıfır-kesinti dağıtım · tenant sayısının job+sorgu yükünü ayrı worker'lara bölmeyi gerektirmesi.

11. v1 kapsamı

İÇİ (ekranlar)

- Login (Keycloak)
- Tenant / kullanıcı / rol yönetimi
- Bağlantılar (CRUD + Test Et)
- Veri haritası tasarımı (tablo seçimi, dostça ad, ilişki, boyut/ölçü, mod)
- Sürükle-bırak sorgu tasarımcısı + sonuç grid'i
- Ham SQL ekranı (Monaco)
- Rapor kaydetme / klasörleme / yetki
- Grafikler
- Dashboard tasarlayıcı + görüntüleyici
- İçe alım job'ları (zamanlama + durum)
- Audit ekranı
- Excel/CSV export

DIŞI (bilinçli)

- Çoklu motor (kapı açık — §5)
- KDS Ajanı / gateway (§3)
- Artımlı yükleme
- Zamanlanmış rapor e-postası
- Rapor embedding / dış API
- Doğal dille sorgu (LLM)
- PDF export
- On-prem
- Mobil uygulama

12. Başarı ölçütleri (v1)

1. SQL bilmeyen bir kullanıcı, iki tablolu join'li + gruplu + filtreli bir raporu **hiç SQL yazmadan** tasarlayıp grafiğe bağlayabilmeli ve dashboard'a koyabilmeli.
2. Canlı modda 1M satırlık kaynak tabloda tasarımcı sorgusu ilk sayfayı < 3 sn'de getirmeli; hiçbir sorgu satır sınırı/zaman aşımı olmadan koşamamalı.
3. İçe alım job'ı zamanında koşup aynı raporu tenant deposundan aynı sonuçla vermeli; mod geçişi (canlı ↔ içe alım) raporu bozmamalı.
4. SELECT dışı hiçbir komut, hiçbir ekrandan/yoldan çalıştırılamamalı — **test kanıtlı** (bekçi + read-only kullanıcı iki katman ayrı ayrı doğrulanır).

13. Riskler

Risk	Etki	Önlem
SaaS → müşteri DB erişimi (firewall)	Satış/kurulum engeli	v1 önkoşul belgesi (erişilebilir DB); Mod B'yi öne çıkar; ajan araştırması kuyrukta

Risk	Etki	Önlem
Sorgu tasarımcısı UX zorluğu	Ürünün kalbi; kötüyse ürün yok	S1'de ekran tasarımına ayrı efor; erken elle doğrulama; rakip inceleme (Metabase/Power BI)
SQL üretici karmaşıklığı	Hatalı/yavaş SQL	v1 model kapsamı sınırlı (§5); altın-örnek (golden master) test seti; "SQL'i Göster" şeffaflığı
Büyük sonuç kümeleri	Bellek/ağ çökmesi	Reader'da satır sınırı + server-side paging + sanallaştırılmış grid
Lisans tuzakları	Ticari sürpriz	§9 tablosu bağlayıcı; her yeni paket lisans denetiminden geçer (SQLST kuralı)
Tenant veri sızıntısı	En ağır güven kaybı	`TenantId` global filter + tenant-başına depo DB + audit; S3'te test kanıtı